Practical 6:

Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

Code:-

events.json:-

```
[
  {
    "id": 1,
    "title": "AI Workshop",
    "category": "Workshop",
    "date": "2026-10-10",
    "location": "CHARUSAT"
  },
  {
    "id": 2,
    "title": "Hackathon 2026",
    "category": "Hackathon",
    "date": "2026-10-15",
    "location": "CHARUSAT"
  },
```

```
{
    "id": 3,
    "title": "Coding Competition",
    "category": "Competition",
    "date": "2026-10-20",
    "location": "Computer Lab"
},
{
    "id": 4,
    "title": "Web Development Seminar",
    "category": "Seminar",
    "date": "2026-10-25",
    "location": "Seminar Hall"
},
{
    "id": 5,
    "title": "Cyber Security Workshop",
    "category": "Workshop",
    "date": "2026-11-02",
    "location": "CHARUSAT"
},
```

```
  {
    "id": 6,
    "title": "Python Programming Contest",
    "category": "Competition",
    "date": "2026-11-10",
    "location": "Computer Lab"
  }
]
```


students.json:-


```
[
  {
    "id": 1,
    "name": "Dharm Solanki",
    "email": "dharm@example.com",
    "course": "B.Tech Computer Engineering",
    "semester": 3
  },
  {
    "id": 2,
    "name": "Priyaba Jadeja",
    "email": "priyaba@example.com",
```

```
    "course": "B.Tech Computer Engineering",
    "semester": 3
  },
  {
    "id": 3,
    "name": "Rahul Patel",
    "email": "rahul@example.com",
    "course": "B.Tech Information Technology",
    "semester": 4
  },
  {
    "id": 4,
    "name": "Amit Shah",
    "email": "amit@example.com",
    "course": "B.Tech Computer Engineering",
    "semester": 5
  },
  {
    "id": 5,
    "name": "Neha Patel",
    "email": "neha@example.com",
```

```
    "course": "B.Tech Information Technology",
    "semester": 3
  }
]
```


notices.json


```
[
  {
    "id": 1,
    "title": "Mid Semester Examination",
    "date": "2026-10-08",
    "message": "Mid semester examination will start from 8 October 2026."
  },
  {
    "id": 2,
    "title": "Hackathon Registration",
    "date": "2026-10-12",
    "message": "Students can register for the upcoming hackathon."
  },
  {
```

"id": 3,

"title": "Holiday Notice",

"date": "2026-10-18",

"message": "The university will remain closed on the announced holiday."

},

{

"id": 4,

"title": "Assignment Submission",

"date": "2026-10-22",

"message": "Submit your assignments before the given deadline."

}

]

FAQ.JSON:-

[

{

"id": 1,

"question": "How can I register for an event?",

"answer": "Open the event section and complete the registration form."

},

{

"id": 2,

"question": "Where can I see university notices?",

"answer": "All important notices are displayed in the Notices section."

},

{

"id": 3,

"question": "How can I contact the college?",

"answer": "You can contact the college office through the official contact details."

},

{

"id": 4,

"question": "How can I view student profiles?",

"answer": "Open the Student Profiles section to view student information."

}

]

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Information Portal</title>

  <style>
    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      font-family: Arial, sans-serif;
      background: #f4f6f8;
      color: #222;
    }

    header {
      background: #1f4e79;
      color: white;
```

```
    padding: 20px;
    text-align: center;
}

nav {
    background: white;
    padding: 15px;
    text-align: center;
    box-shadow: 0 2px 5px #ccc;
}

nav button {
    padding: 10px 18px;
    margin: 5px;
    border: none;
    background: #1f4e79;
    color: white;
    border-radius: 5px;
    cursor: pointer;
}
```

```
nav button:hover {
    background: #163a5a;
}

.container {
    width: 90%;
    max-width: 1100px;
    margin: 25px auto;
}

.controls {
    background: white;
    padding: 20px;
    margin-bottom: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 5px #ddd;
}

input,
select {
    padding: 10px;
```

```
        margin: 5px;
        border: 1px solid #aaa;
        border-radius: 5px;
    }

    input {
        width: 250px;
    }

    .card-container {
        display: grid;
        grid-template-columns: repeat(auto-fit,
minmax(280px, 1fr));
        gap: 20px;
    }

    .card {
        background: white;
        padding: 20px;
        border-radius: 8px;
        box-shadow: 0 2px 6px #ddd;
```

```
}

.card h3 {
    color: #1f4e79;
    margin-top: 0;
}

.card p {
    line-height: 1.6;
}

.pagination {
    text-align: center;
    margin: 25px 0;
}

.pagination button {
    padding: 8px 14px;
    margin: 3px;
    border: none;
    background: #1f4e79;
```

```
    color: white;
    border-radius: 4px;
    cursor: pointer;
}

.pagination button.active {
    background: #e67e22;
}

.pagination button:disabled {
    background: #aaa;
    cursor: not-allowed;
}

.hidden {
    display: none;
}

.faq {
    cursor: pointer;
}
```

```
.answer {
    display: none;
    background: #f1f1f1;
    padding: 10px;
    margin-top: 10px;
    border-radius: 5px;
}

footer {
    background: #1f4e79;
    color: white;
    text-align: center;
    padding: 15px;
    margin-top: 40px;
}

@media (max-width: 600px) {
    input,
    select {
        width: 100%;
```

```
            margin: 5px 0;
        }

        nav button {
            width: 90%;
        }

        .container {
            width: 95%;
        }
    }
  </style>
</head>

<body>

<header>
  <h1>Student Information Portal</h1>
  <p>Dynamic Data using Fetch API and JSON</p>
</header>
```

```
<nav>
  <button onclick="showSection('events')">Events</button>
  <button onclick="showSection('students')">Students</button>
  <button onclick="showSection('notices')">Notices</button>
  <button onclick="showSection('faqs')">FAQs</button>
</nav>

<div class="container">

  <div id="events">
    <h2>Event List</h2>

    <div class="controls">
      <input
        type="text"
        id="eventSearch"
        placeholder="Search event..."
        onkeyup="filterEvents()"
      >
```

```
        <select id="eventCategory" onchange="filterEvents()">
          <option value="all">All Categories</option>
          <option value="Workshop">Workshop</option>
          <option value="Hackathon">Hackathon</option>
          <option
value="Competition">Competition</option>
          <option value="Seminar">Seminar</option>
        </select>

        <select id="eventSort" onchange="filterEvents()">
          <option value="default">Sort By</option>
          <option value="nameAsc">Name A-Z</option>
          <option value="nameDesc">Name Z-A</option>
          <option value="dateAsc">Date Old-New</option>
          <option value="dateDesc">Date New-Old</option>
        </select>
      </div>

      <div id="eventList" class="card-container"></div>
```

```
      <div id="eventPagination" class="pagination"></div>
    </div>


    <div id="students" class="hidden">
      <h2>Student Profiles</h2>

      <div class="controls">
        <input
          type="text"
          id="studentSearch"
          placeholder="Search student..."
          onkeyup="filterStudents()"
        >

        <select id="studentSemester"
onchange="filterStudents()">
          <option value="all">All Semesters</option>
          <option value="3">Semester 3</option>
          <option value="4">Semester 4</option>
          <option value="5">Semester 5</option>
```

```
        </select>

        <select id="studentSort" onchange="filterStudents()">
          <option value="default">Sort By</option>
          <option value="nameAsc">Name A-Z</option>
          <option value="nameDesc">Name Z-A</option>
          <option value="semesterAsc">Semester Low-
High</option>
          <option value="semesterDesc">Semester High-
Low</option>
        </select>
      </div>

      <div id="studentList" class="card-container"></div>

      <div id="studentPagination" class="pagination"></div>
    </div>


    <div id="notices" class="hidden">
      <h2>University Notices</h2>
```

```
<div class="controls">
  <input
    type="text"
    id="noticeSearch"
    placeholder="Search notice..."
    onkeyup="filterNotices()"
  >

  <select id="noticeSort" onchange="filterNotices()">
    <option value="default">Sort By</option>
    <option value="dateAsc">Oldest First</option>
    <option value="dateDesc">Newest First</option>
    <option value="nameAsc">Title A-Z</option>
    <option value="nameDesc">Title Z-A</option>
  </select>
</div>

<div id="noticeList" class="card-container"></div>

<div id="noticePagination" class="pagination"></div>
```

```
    </div>


    <div id="faqs" class="hidden">
        <h2>Frequently Asked Questions</h2>

        <div class="controls">
            <input
                type="text"
                id="faqSearch"
                placeholder="Search FAQ..."
                onkeyup="filterFAQs()"
            >
        </div>

        <div id="faqList" class="card-container"></div>

        <div id="faqPagination" class="pagination"></div>
    </div>

</div>
```

```
<footer>
   Student Information Portal
</footer>

<script src="script.js"></script>

</body>
</html>
```

script.js

```
let events = [];
let students = [];
let notices = [];
let faqs = [];

let eventPage = 1;
let studentPage = 1;
let noticePage = 1;
let faqPage = 1;

const itemsPerPage = 3;
```

```
fetch("events.json")
    .then(response => response.json())
    .then(data => {
        events = data;
        filterEvents();
    });

fetch("students.json")
    .then(response => response.json())
    .then(data => {
        students = data;
        filterStudents();
    });

fetch("notices.json")
    .then(response => response.json())
    .then(data => {
        notices = data;
        filterNotices();
    });
```

```
fetch("faqs.json")
    .then(response => response.json())
    .then(data => {
        faqs = data;
        filterFAQs();
    });


function showSection(section) {

document.getElementById("events").classList.add("hidden");

document.getElementById("students").classList.add("hidden"
);

document.getElementById("notices").classList.add("hidden");
    document.getElementById("faqs").classList.add("hidden");


document.getElementById(section).classList.remove("hidden
");
```

```
}



function filterEvents() {
   let search =
document.getElementById("eventSearch").value.toLowerCas
e();
   let category =
document.getElementById("eventCategory").value;
   let sort = document.getElementById("eventSort").value;

   let result = events.filter(event => {
      let searchMatch =
         event.title.toLowerCase().includes(search) ||
         event.location.toLowerCase().includes(search);

      let categoryMatch =
         category === "all" ||
         event.category === category;

      return searchMatch && categoryMatch;
```

```
    });

    if (sort === "nameAsc") {
        result.sort((a, b) => a.title.localeCompare(b.title));
    }

    if (sort === "nameDesc") {
        result.sort((a, b) => b.title.localeCompare(a.title));
    }

    if (sort === "dateAsc") {
        result.sort((a, b) => new Date(a.date) - new
Date(b.date));
    }

    if (sort === "dateDesc") {
        result.sort((a, b) => new Date(b.date) - new
Date(a.date));
    }

    displayEvents(result);
```

```
}



function displayEvents(data) {
    let start = (eventPage - 1) * itemsPerPage;
    let end = start + itemsPerPage;

    let pageData = data.slice(start, end);

    let output = "";

    pageData.forEach(event => {
        output += `
            <div class="card">
                <h3>${event.title}</h3>
                <p><b>Category:</b> ${event.category}</p>
                <p><b>Date:</b> ${event.date}</p>
                <p><b>Location:</b> ${event.location}</p>
            </div>
        `;
    });
```

```
    if (output === "") {
        output = "<p>No events found.</p>";
    }

    document.getElementById("eventList").innerHTML =
output;

    createPagination(
        data.length,
        eventPage,
        "eventPagination",
        page => {
            eventPage = page;
            displayEvents(data);
        }
    );
}

function filterStudents() {
```

```
    let search =
document.getElementById("studentSearch").value.toLowerCa
se();
    let semester =
document.getElementById("studentSemester").value;
    let sort = document.getElementById("studentSort").value;

    let result = students.filter(student => {
        let searchMatch =
            student.name.toLowerCase().includes(search) ||
            student.email.toLowerCase().includes(search) ||
            student.course.toLowerCase().includes(search);

        let semesterMatch =
            semester === "all" ||
            student.semester == semester;

        return searchMatch && semesterMatch;
    });

    if (sort === "nameAsc") {
```

```
        result.sort((a, b) => a.name.localeCompare(b.name));
    }

    if (sort === "nameDesc") {
        result.sort((a, b) => b.name.localeCompare(a.name));
    }

    if (sort === "semesterAsc") {
        result.sort((a, b) => a.semester - b.semester);
    }

    if (sort === "semesterDesc") {
        result.sort((a, b) => b.semester - a.semester);
    }

    displayStudents(result);
}


function displayStudents(data) {
    let start = (studentPage - 1) * itemsPerPage;
```

```
let end = start + itemsPerPage;

let pageData = data.slice(start, end);

let output = "";

pageData.forEach(student => {
    output += `
        <div class="card">
            <h3>${student.name}</h3>
            <p><b>Email:</b> ${student.email}</p>
            <p><b>Course:</b> ${student.course}</p>
            <p><b>Semester:</b> ${student.semester}</p>
        </div>
    `;
});

if (output === "") {
    output = "<p>No students found.</p>";
}
```

```
    document.getElementById("studentList").innerHTML =
output;


    createPagination(
        data.length,
        studentPage,
        "studentPagination",
        page => {
            studentPage = page;
            displayStudents(data);
        }
    );
}



function filterNotices() {
    let search =
document.getElementById("noticeSearch").value.toLowerCas
e();
    let sort = document.getElementById("noticeSort").value;
```

```
  let result = notices.filter(notice => {
    return (
      notice.title.toLowerCase().includes(search) ||
      notice.message.toLowerCase().includes(search)
    );
  });

  if (sort === "dateAsc") {
    result.sort((a, b) => new Date(a.date) - new
Date(b.date));
  }

  if (sort === "dateDesc") {
    result.sort((a, b) => new Date(b.date) - new
Date(a.date));
  }

  if (sort === "nameAsc") {
    result.sort((a, b) => a.title.localeCompare(b.title));
  }
```

```
    if (sort === "nameDesc") {
        result.sort((a, b) => b.title.localeCompare(a.title));
    }

    displayNotices(result);
}


function displayNotices(data) {
    let start = (noticePage - 1) * itemsPerPage;
    let end = start + itemsPerPage;

    let pageData = data.slice(start, end);

    let output = "";

    pageData.forEach(notice => {
        output += `
            <div class="card">
                <h3>${notice.title}</h3>
                <p><b>Date:</b> ${notice.date}</p>
```

```
            <p>${notice.message}</p>
        </div>
    `;
});

if (output === "") {
    output = "<p>No notices found.</p>";
}

document.getElementById("noticeList").innerHTML =
output;

createPagination(
    data.length,
    noticePage,
    "noticePagination",
    page => {
        noticePage = page;
        displayNotices(data);
    }
);
```

```
}


function filterFAQs() {
   let search =
document.getElementById("faqSearch").value.toLowerCase();

   let result = faqs.filter(faq => {
      return (
         faq.question.toLowerCase().includes(search) ||
         faq.answer.toLowerCase().includes(search)
      );
   });

   displayFAQs(result);
}


function displayFAQs(data) {
   let start = (faqPage - 1) * itemsPerPage;
   let end = start + itemsPerPage;
```

```
    let pageData = data.slice(start, end);

    let output = "";

    pageData.forEach(faq => {
        output += `
            <div class="card faq"
onclick="toggleAnswer(${faq.id})">
                <h3>${faq.question}</h3>
                <div id="answer-${faq.id}" class="answer">
                    ${faq.answer}
                </div>
            </div>
        `;
    });

    if (output === "") {
        output = "<p>No FAQs found.</p>";
    }
```

```
    document.getElementById("faqList").innerHTML = output;

    createPagination(
        data.length,
        faqPage,
        "faqPagination",
        page => {
            faqPage = page;
            displayFAQs(data);
        }
    );
}


function toggleAnswer(id) {
    let answer = document.getElementById("answer-" + id);

    if (answer.style.display === "block") {
        answer.style.display = "none";
    } else {
        answer.style.display = "block";
```

```
    }
}



function createPagination(totalItems, currentPage,
elementId, callback) {
    let totalPages = Math.ceil(totalItems / itemsPerPage);

    let output = "";

    if (totalPages <= 1) {
        document.getElementById(elementId).innerHTML = "";
        return;
    }

    output += `
        <button
            onclick="paginationAction(${currentPage - 1})"
            ${currentPage === 1 ? "disabled" : ""}
        >
            Previous
```

```
    </button>
  `;

  for (let i = 1; i <= totalPages; i++) {
    output += `
      <button
        class="${i === currentPage ? "active" : ""}"
        onclick="paginationAction(${i})"
      >
        ${i}
      </button>
    `;
  }

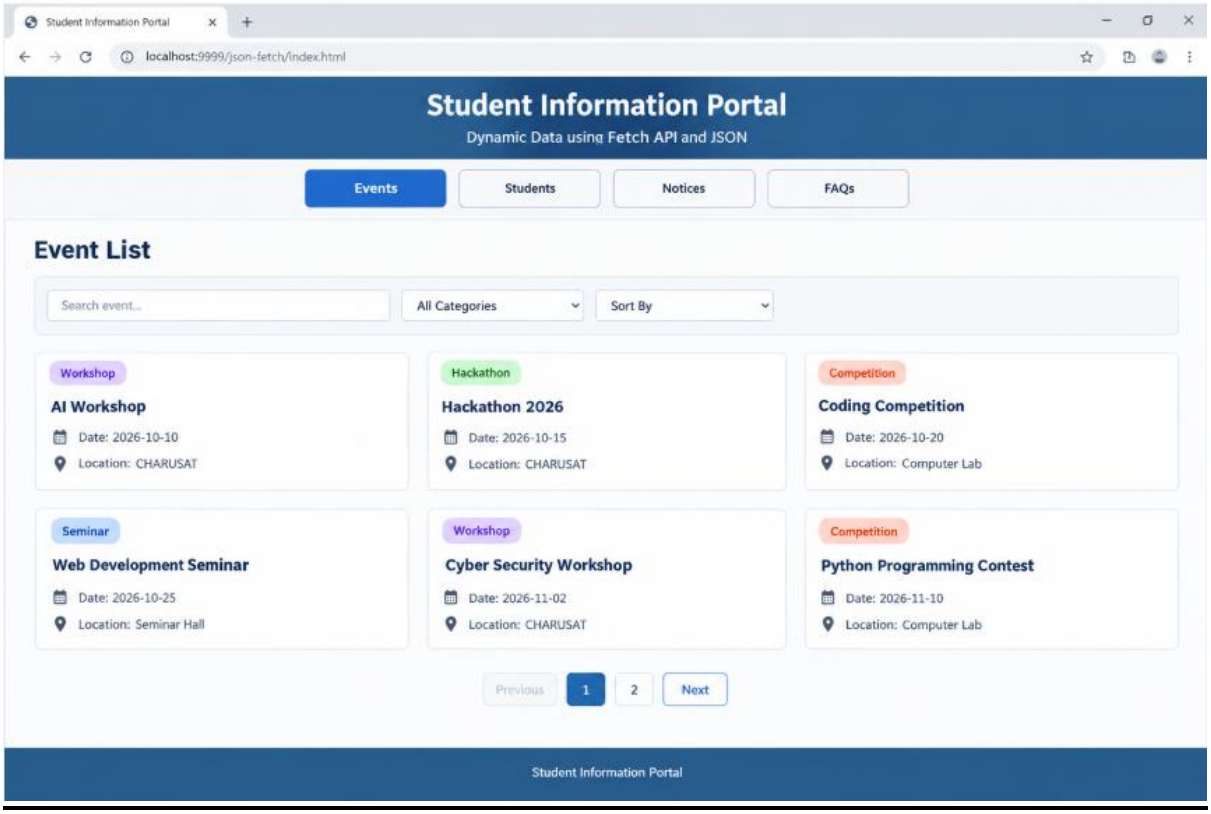
  output += `
    <button
      onclick="paginationAction(${currentPage + 1})"
      ${currentPage === totalPages ? "disabled" : ""}
    >
      Next
    </button>
```

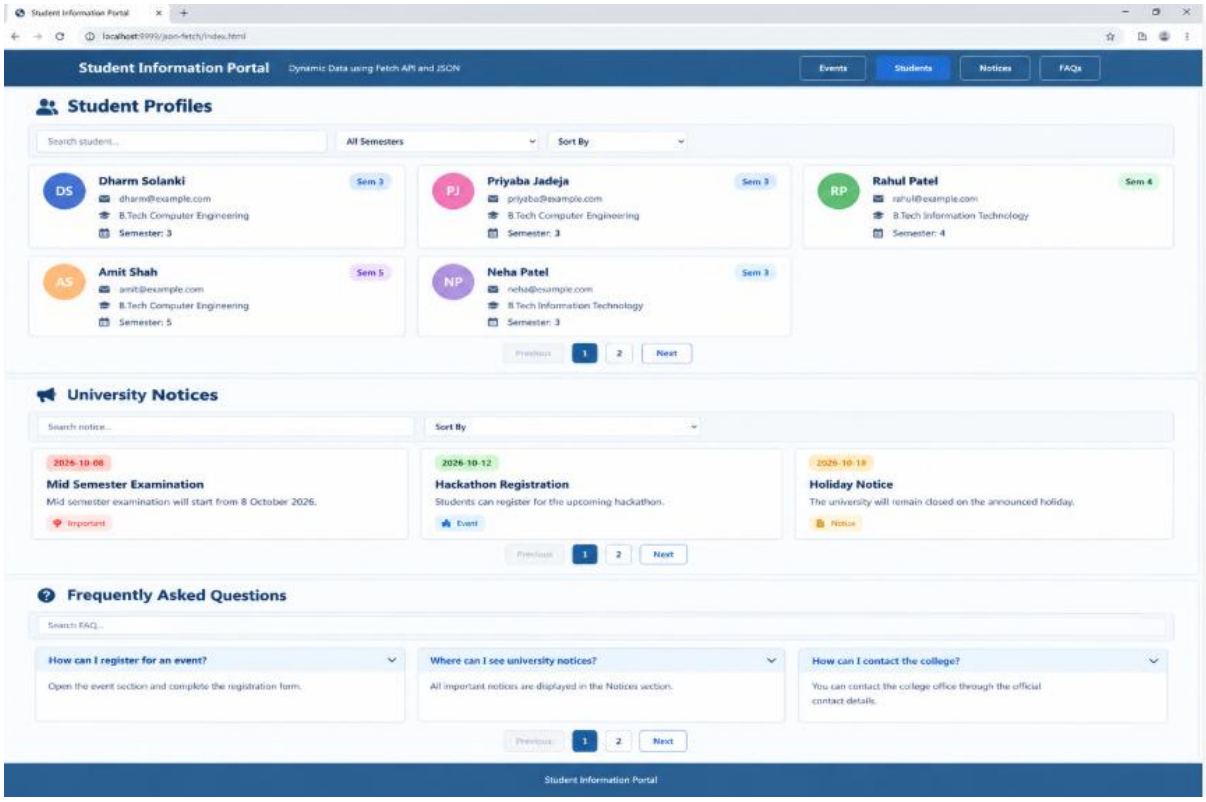
```
    `;

    document.getElementById(elementId).innerHTML =
output;

    window.paginationAction = callback;
}
```

OUTPUT:-





Key Questions / Analysis / Interpretation

1. How is JSON fetched, parsed, and rendered?

JSON data is fetched using the Fetch API. The response.json() method converts the JSON response into JavaScript objects/arrays. The data is then rendered dynamically using innerHTML.

2. Which array methods are used for search, filter, sort, and pagination?

- Search/Filter: filter()
- Sorting: sort()
- Pagination: slice()
- Display: forEach()

3. How are loading and error states handled?

The current implementation does not explicitly handle loading and error states. It directly fetches the JSON files. catch() can be added to display an error message if the JSON file cannot be loaded.

4. How is modularity maintained in JavaScript files?

The JavaScript is kept in a separate script.js file instead of writing JavaScript inside HTML. Different functions are created for events, students, notices, FAQs, filtering, sorting, rendering, and pagination, making the code easier to maintain.

Tools/Technology to be Used :-

JavaScript ES6+, JSON, HTML5, CSS3,VS CODE